Stable Diffusion Api For Generating Images

Authors: Gyan Kannur

Assignment Week

Use the Stable Diffusion API within a Jupyter Notebook to generate and edit an image. Be sure to include Markdown that describes what you're doing and a narrative on the result and your thoughts on how easy or hard the API was to use.

Api Keys

Importing data

First, we read the api keys stored in a file and then set the environment variable.

```
import asyncio
import aiohttp
import os
from IPython.display import Image, display

def _set_env_from_file(var: str, file_path: str = r"C:\Users\gyanr\
gyan-python-workspace\grk_langchain\app\notebooks\data\
stability_key.txt"):
    """
    Reads an API key from a specified file and sets it as an
environment variable.
    """
    if not os.environ.get(var):
        try:
            # The 'with open' statement ensures the file is closed
automatically
            with open(file_path, 'r') as f:
                # Read the first line and strip any leading/trailing
whitespace
                key = f.readline().strip()

            if key:
                os.environ[var] = key
                print(f"Successfully loaded {var} from {file_path}")
            else:
                print(f"Warning: {file_path} is empty.")

        except FileNotFoundError:
            print(f"Error: Key file not found at {file_path}. Please
create the file.")
```

```
_set_env_from_file('STABILITY_API_KEY')
print(os.environ.get('STABILITY_API_KEY'))
```

```
Successfully loaded STABILITY_API_KEY from C:\Users\gyanr\gyan-python-workspace\grk_langchain\app\notebooks\data\stability_key.txt
sk-TGLbJdvwXLQVbHfchvixc29LzyTehbA7ZayLsZPsYlXZN5Qy
```

Api Documentation on Text To Image Endpoint

Stability AI's official API

documentation available https://platform.stability.ai/docs/legacy/grpc-api/features/text-to-image#Python They also provide stability sdk package or using an api

```
engine_id = "stable-diffusion-xl-1024-v1-0"
url = "https://stablediffusionapi.com/api/v3/text2img"
api_key = os.getenv("STABILITY_API_KEY")
api_host = "https://api.stability.ai"

if api_key is None:
    raise Exception("Missing Stability API key.")
```

API endpoint changes

The v2beta API only supports multipart/form-data — there's no JSON option on the new endpoints. However, the old v1 endpoint you showed does accept JSON.

```
async def generate_image(user_prompt: str) -> list[bytes]:
    url = f"{api_host}/v2beta/stable-image/generate/core"

    headers = {
        "Authorization": f"Bearer {api_key}",
        "Accept": "image/*"
    }

    data = aiohttp.FormData()
    data.add_field("prompt", user_prompt)
    data.add_field("output_format", "png", content_type="text/plain")
    data.add_field("aspect_ratio", "1:1", content_type="text/plain")

    async with aiohttp.ClientSession() as session:
        async with session.post(url, headers=headers, data=data) as
response:
            if response.status != 200:
                text = await response.text()
                raise Exception(f"Non-200 response: {text}")

            print(f"Status: {response.status}")
```

```
            return await response.read()


# user_prompt = input("Enter your prompt: ")
user_prompt = "Little Girl chasing butterfly with a cat running
behind."
result = await generate_image(user_prompt)

display(Image(result))
```

```
Status: 200
```

Api specs for Image Size

In the Stability AI v2beta REST API (used for Stable Image Ultra, Core, and SD3), the system is designed to always output a 1-megapixel image. Because these models are specifically tuned for that resolution, you cannot simply pass a "width" or "height" to make the image smaller in terms of pixels like you could in the older V1 API.

Now lets create an image directory

```
image_dir = os.path.join(os.path.curdir, "stabilityio/images/")
if os.path.exists(image_dir):
    pass
else:
    os.makedirs(image_dir)


def save_images(byt_images: bytes, image_name: str = "image") -> str:

    filename=f"{image_dir}/{image_name}.png"

    with open(filename,"wb") as f:
        f.write(byt_images)

    print(f"Saved image {filename}")
    return filename


file_path = save_images(result, image_name="girl_cat")

# file_path = r"C:\Users\gyanr\gyan-python-workspace\generative-ai\
assignments\week-3\stabilityio\images\test.png"
# display(Image(filename=file_path))
```

Edit image functionality

stable-image/edit/search-and-replace api for edit

For now I am planning to write a function which takes in the image as bytes and the edit prompt and search_for parameters which are part of the api specs as function parameters.

```
async def edit_image(image_bytes: bytes, edit_prompt: str, search_for:
str) -> bytes:
    url = f"{api_host}/v2beta/stable-image/edit/search-and-replace"
    print(url)

    headers = {
        "Authorization": f"Bearer {api_key}",
        "Accept": "image/*",
        "stability-client-id": "my-app"
    }
```

```
    form = aiohttp.FormData()
    form.add_field("image", image_bytes,
                   content_type="image/png",
                   filename="image.png")
    form.add_field("prompt", edit_prompt, content_type="text/plain")
# what to replace WITH
    form.add_field("search_prompt", search_for,
content_type="text/plain") # what to search FOR
    form.add_field("output_format", "png", content_type="text/plain")

    async with aiohttp.ClientSession() as session:
        async with session.post(url, headers=headers, data=form) as
response:
            if response.status != 200:
                text = await response.text()
                raise Exception(f"Non-200 response: {text}")

            print(f"Status: {response.status}")
            return await response.read()

image_bytes=None
with open(file_path, "rb") as f:
    image_bytes = f.read()

edited_bytes = await edit_image(image_bytes, "replace the cat with a
dog", search_for="cat")
save_images(edited_bytes, image_name="girl_dog")
display(Image(filename=file_path))

async  def
call_edit_for_image(given_file,new_file,given_prompt,src_prompt):
    image_bytes=None
    with open(given_file, "rb") as f:
        image_bytes = f.read()

    edited_bytes = await edit_image(image_bytes, given_prompt,
search_for=src_prompt)
    img_res =save_images(edited_bytes, image_name=new_file)
    return img_res



edited_bytes = await edit_image(image_bytes, "a golden retriever dog",
search_for="cat")
save_images(edited_bytes, image_name="girl_with_dog")
display(Image(filename=file_path))
```

```
https://api.stability.ai/v2beta/stable-image/edit/search-and-replace
Status: 200
Saved image .\stabilityio/images//girl_with_dog.png
```

Edited Image Result

Clearly this is a typical case of hallucination and more better prompting requirements. The ask was to search for a cat and replace that with a dog. The result was one of the cat being replaced with a brown cat. Let me try with a different prompt.

Python code changes

I created some common functions to avoid repetation

```
file_path = r"C:\Users\gyanr\gyan-python-workspace\generative-ai\
assignments\week-3\stabilityio\images\girl_cat.png"
```

```
result2=await
call_edit_for_image(given_file=file_path,new_file="cat_with_dog",given
_prompt="A golden retriever running",src_prompt="girl")
display(Image(filename=result2))
```

```
https://api.stability.ai/v2beta/stable-image/edit/search-and-replace
Status: 200
Saved image .\stabilityio/images//cat_with_dog.png
```

Result

Finally the girl was replaced with a dog.

Summary and Challenges

Challenges: I had a problem with the right diffusion API platform; apparently, there were two sites with almost identical names providing similar functionality. I observed that some of the APIs, which were in beta, did not support JSON, and hence I had to pass parameters as form fields. It was fun learning asyncio instead of calling the mundane REST Python API.

Summary: Whether you use an API or a GUI, the hallucinations and prompting techniques are more or less the same. The challenge with the API was ensuring that the right data and string types were passed. Overall, it is a very solid API for image generation.